

Security Assessment and Remediation Report

SE4030 — Secure Software Development

Student	Amila Devin Senarathne — <<YOUR INDEX NUMBER>>
Application	Instagram-Clone (MERN Stack) by Anass Ferrak ("TheLordA")
Original repository	https://github.com/TheLordA/Instagram-Clone
Modified repository	https://github.com/amilal998/SE4030-Assignment---Instagram-Clone-master
Last upstream commit	ddc1b1e — 9 March 2022 (predates the 1 July 2026 semester start)
Video	<<YOUTUBE LINK>>

Executive summary

This report documents a security assessment of an existing open-source MERN application, the remediation of every issue found, and the addition of an OAuth 2.0 / OpenID Connect sign-in flow.

Twelve distinct vulnerabilities were identified using a combination of white-box (static analysis, dependency scanning, manual code review) and black-box (live HTTP probing) techniques. All twelve were fixed, each as a single self-contained commit.

Two findings were **critical**:

1. **VULN-02** — live password-reset tokens were returned by a routine profile endpoint, allowing any user to take over any account in three HTTP requests.
2. **VULN-03** — NoSQL operator injection in every authentication query, the worst instance allowing a password reset against an account the attacker had never identified.

These two **chain**: the unauthenticated user-search endpoint (VULN-01) supplied the victim's internal ID, which VULN-02 turned into a reset token. Pre-authentication, an attacker could compromise arbitrary accounts.

One further finding, **VULN-11**, is notable because it was a defect *in one of our own fixes*: the rate limiter added for VULN-07 was completely inert because the application returned HTTP 200 on failed logins. It was caught only because the fixes were tested against a running server rather than assumed correct from reading the diff.

Metric	Before	After
Server dependency advisories	32 (2 critical, 17 high)	0
Client dependency advisories	200 (18 critical, 59 high)	26 (build-toolchain only)
Custom Semgrep findings	17	0
Runtime security checks passing	—	30 / 30
OAuth flow checks passing	—	23 / 23

1. Application selected

1.1 Why this application

Criterion	Assessment
Not a teaching-vulnerability app	Correct — a genuine social application with 130+ stars, no planted flaws
Last commit predates semester	Yes — <code>ddc1b1e</code> , 9 March 2022, against a 1 July 2026 semester start (see §1.2)
Sufficient scope and complexity	Full-stack MERN: 3 controllers, 17 API endpoints, 2 Mongoose models, 9 React screens, JWT auth, image upload, password reset by email
Improved version not public	No security-hardened fork exists
OAuth is a genuine change	It had only homegrown email/password auth, so adding Google sign-in is a real feature, not a duplicate

The vulnerabilities found are **ordinary developer mistakes** — a lower-case `p` in a property name, a missing middleware argument, a deny-list where an allow-list was needed. That is precisely what makes the exercise realistic: these are the bugs that reach production, not contrived exercises.

1.2 Provenance and eligibility

The assignment requires the original project's last commit to predate the start of the semester (**1 July 2026**). That was verified against the GitHub API rather than assumed:

Evidence	Value
Default branch (<code>master</code>) tip	<code>ddc1b1e</code>
Author date of that commit	2022-03-09T15:18:47Z
Newest commit on <i>any</i> branch	<code>14fccf6</code> , 2022-04-04 (an abandoned Dependabot branch, never merged)
Repository <code>pushed_at</code> metadata	2023-09-24

A caution on that last row: the repository's `pushed_at` field reads September 2023, but **no commit was authored then**. That timestamp records a ref being pushed, not new work; the newest commit object in the repository is from April 2022. Quoting `pushed_at` as the "last commit date" would therefore be incorrect, so the authored date of the commit actually used is cited instead.

The baseline in this repository was confirmed to be that exact upstream commit, not an approximation of it:

```
$ git diff --stat <baseline> upstream/master -- server client
$                               # empty: byte-identical across both application trees
```

The margin is **over four years**, so eligibility does not depend on which of the dates above a marker prefers.

1.3 Architecture

React SPA (:3000) —HTTP + JWT Bearer→ Express API (:5000) → MongoDB

Authentication was a homegrown flow: bcrypt-hashed passwords, a JWT returned on login, stored in `localStorage`, sent as `Authorization: Bearer`.

2. Methodology and tooling

A defect class missed by one technique is usually caught by another, so four were layered.

Technique	Tool	What it found
SCA (white-box)	<code>npm audit</code>	VULN-10, VULN-12 — 232 advisories across both trees
SAST (white-box)	Semgrep — community rulesets	Nothing (0 findings)
SAST (white-box)	Semgrep — custom ruleset	VULN-01 to VULN-06, VULN-09 — 17 findings
Manual review (white-box)	—	VULN-06, VULN-07, VULN-08
Runtime probing (black-box)	<code>verify-fixes.js</code> , <code>curl</code>	VULN-11, and confirmation of all fixes

2.1 The most important methodological finding

The Semgrep community rulesets found nothing. Running `p/javascript`, `p/nodejs`, `p/react` and `p/security-audit` over the codebase produced **0 findings** — while the application contained two critical vulnerabilities.

This is worth stating plainly because it is the single most transferable lesson of the assessment: *a clean SAST report is not evidence of a secure application*. Generic rulesets look for recognised dangerous **patterns** (`eval`, string-concatenated SQL, `dangerouslySetInnerHTML`). The defects here were failures of application **logic** — a route missing an argument, a query trusting the type of its input, a projection that was a deny-list instead of an allow-list. No generic rule models "this Express route has one fewer argument than every other route in the file". So a custom ruleset was written (`security/semgrep-rules.yml`, 7 rules) encoding the specific properties this codebase should hold — the same code, the same tool, different rules:

Rule	Findings on the vulnerable baseline
<code>raw-error-object-in-response</code>	10
<code>secret-schema-field-not-excluded-by-default</code>	2
<code>nosql-operator-injection-from-request</code>	1
<code>express-route-missing-auth-middleware</code>	1
<code>jwt-signed-without-expiry</code>	1
<code>jwt-verify-without-algorithm-pinning</code>	1
<code>regex-built-from-user-input</code>	1
Total	17

The negative result is kept in `security/reports/semgrep-community-before.txt` deliberately, as evidence for this conclusion.

2.2 Tuning the rules — and why the count changed

The first version of these rules reported **20** findings. Re-running them against the *fixed* code surfaced 5 findings that turned out to be **false positives** — artefacts of how the rules were written, not defects:

False positive	Cause
<code>req.params.id</code> flagged as injectable	A URL path segment is always a string — an object cannot be expressed there — so <code>req.params</code> is not an operator-injection vector. <code>req.query</code> is (<code>?a[\$ne]=1</code>), and remains in scope.
<code>jwt.verify(t, s, verifyOptions)</code> flagged as unpinned	The exclusion only recognised an inline object literal . Extracting the options into a shared config module — the better practice the rule exists to encourage — made the rule fire on its own remediation.
<code>Password: { ..., select: false }</code> still flagged	Semgrep matches object patterns partially , so the pattern also matched fields that already carried the fix.

The rules were corrected, which is why the baseline figure in this report is **17** rather than 20: three of the original hits were `req.params` non-issues, and the other two false positives were on the *fixed* code.

This was then regression-tested rather than assumed: the **tuned** rules were re-run against the original baseline commit in a separate `git worktree` and still produce **17 findings with all seven rules firing**, confirming that removing the false positives did not blind any rule to the real defects.

A rule that fires on its own remediation trains people to ignore it. Tuning out false positives is therefore part of the fix, not housekeeping — an alerting channel that cries wolf is worse than no channel, because it consumes attention and teaches dismissal.

Result on the fixed code: 0 findings (`security/reports/semgrep-custom-after.txt`).

3. Vulnerabilities

Severity uses CVSS-style qualitative bands. Each entry gives the flaw, its impact, the fix, and how the fix was verified.

VULN-01 — Unauthenticated user-search endpoint

OWASP	A01:2021 Broken Access Control
CWE	CWE-306: Missing Authentication for Critical Function
Severity	High
Found by	Custom Semgrep rule <code>express-route-missing-auth-middleware</code>

The flaw. Every route in the application attaches `loginmiddleware` except one:

```
app.post("/users-research", controller.userSearch); // no middleware
```

Impact. `userSearch` returns `_id`, `Email` and `Name` for every match, and an empty pattern matches everything:

```
curl -X POST http://localhost:5000/users-research \
-H 'Content-Type: application/json' -d '{"pattern":""}'
```

That is the complete user table — names, email addresses and internal ObjectIds — to an anonymous caller. The emails feed credential stuffing and phishing; the ObjectIds are the input to VULN-02.

The fix. Attach the existing middleware:

```
app.post("/users-research", loginmiddleware, controller.userSearch);
```

Verified. `POST /users-research` without a token returns `401`.

VULN-02 — Password-reset tokens disclosed → account takeover

OWASP	A01:2021 Broken Access Control / A02:2021 Cryptographic Failures
CWE	CWE-200: Exposure of Sensitive Information
Severity	CRITICAL
Found by	Custom Semgrep rule <code>secret-schema-field-not-excluded-by-default</code>

The flaw. The profile endpoint used a **deny-list** projection:

```
User.findOne({ _id: req.params.id }).select("-Password")
```

That removes `Password` — and nothing else. The schema also stores `ResetToken` and `ExpirationToken`, the secrets guarding the password-reset flow, as ordinary selected-by-default fields. Both were serialised into the response.

Impact — full account takeover in three requests:

1. `POST /reset-pwd` with the victim's email. The server writes a fresh `ResetToken` onto the victim's document. (*The attacker never needs the victim's mailbox.*)
2. `GET /user/<victim id>` — read the live `ResetToken` out of the JSON.
3. `POST /new-pwd` with that token and a password of the attacker's choosing.

Step 2's victim ID came free from VULN-01, so **before authentication** an attacker could take over arbitrary accounts. This defeats authentication entirely, requires no cryptographic attack, and generates no unusual traffic.

The fix. Two independent layers:

1. **Schema** — `Password`, `ResetToken`, `ExpirationToken` are now `select: false`. Secrets became *opt-in* rather than *opt-out*, so a future query that forgets to exclude them cannot leak them.

2. **Controller** — the deny-list became an **allow-list**: `.select("_id Name Email Photo PhotoType Followers Following")`. New schema fields are now private by default.

`signin` is the only path that legitimately needs the hash and opts in with `.select("+Password")`.

The general lesson. A deny-list is wrong by default: it leaks every field somebody forgets to add to it. An allow-list is safe by default — it omits every field somebody forgets to add. Where the failure mode is asymmetric, choose the direction whose failure is harmless.

Verified. `GET /user/:id` response contains neither `ResetToken` nor `Password`, checked after deliberately triggering a reset.

VULN-03 — NoSQL operator injection in every auth query

OWASP	A03:2021 Injection
CWE	CWE-943: Improper Neutralization in Data Query Logic
Severity	CRITICAL
Found by	Custom Semgrep rule <code>nosql-operator-injection-from-request</code>

The flaw. JSON bodies can contain any type, but every authentication query assumed strings and passed the value straight into Mongoose:

```
User.findOne({ Email: email })           // signin, signup
User.findOne({ ResetToken: Token, ... })  // newPwd
```

Send an **object** instead of a string and Mongoose treats it as a query operator, turning an equality test into an attacker-chosen comparison.

Impact, worst first:

Request	Effect
<code>POST /new-pwd {"token": {"\$ne": null}, "password": "x"}</code>	Filter becomes "any user whose ResetToken isn't null". The attacker is handed whichever account has a live reset token and sets its password. No knowledge of the real token needed.
<code>POST /reset-pwd {"email": {"\$ne": null}}</code>	Stamps a reset token onto a victim chosen by the <i>database</i> . Chains into the above.
<code>POST /signin {"email": {"\$ne": null}, ...}</code>	Selects an arbitrary account for the bcrypt comparison — an account-existence oracle. Login is not bypassed, because bcrypt must still match.
<code>POST /signup {"email": {"\$ne": null}}</code>	Duplicate check matches an unrelated user — persistent denial of registration.

The fix. Two layers:

1. **Application-wide** — `express-mongo-sanitize` strips keys beginning `$` or containing `.` from body, query and params, and logs each attempt.
2. **Per-controller type guards** — `signin`, `signup`, `resetPwd` and `newPwd` reject non-strings with `400`.

Why rejecting beats coercing. The tempting one-liner is `String(email)`. But `String({$ne: null})` is `"[object Object]"` — a perfectly valid string that silently changes the query's meaning instead of refusing it. An explicit `typeof !== "string"` check fails *closed* on any unexpected shape, which is strictly stronger.

Verified. `{"$ne":null}` in the email field of `/signin` and the token field of `/new-pwd` both return `400`.

VULN-04 — Regex injection and ReDoS in user search

OWASP	A03:2021 Injection
CWE	CWE-1333 / CWE-200
Severity	High
Found by	Custom Semgrep rule <code>regex-built-from-user-input</code>

The flaw.

```
let pattern = new RegExp("^" + req.body.pattern);
User.find({ Email: { $regex: pattern } })
```

The `^` looks like it constrains the search to a prefix. It constrains nothing — the caller writes everything after it, and metacharacters are interpreted, not matched literally.

Impact.

1. **Bulk PII harvesting.** `{"pattern": ".*"}` returns every user; `{"pattern": ".*@company\\.com"}` returns every employee of one domain.
2. **ReDoS against the database.** The pattern is evaluated by *MongoDB* against every scanned document, so `{"pattern": "(a+)+$"}` burns database CPU shared by all users — from one cheap HTTP request.
3. **No result cap** — a broad prefix returned the entire collection.

The fix. Escape all metacharacters so input is matched literally; require 3–64 characters; cap results at 20.

```
const escapeRegExp = (v) => v.replace(/[.*+?^${}()|[\]\\]/g, "\\$&");
```

Escaping removes the quantifier primitive entirely, which is more robust than trying to *detect* catastrophic patterns.

Verified. `".*"` no longer matches `alice@x.com` (it now matches only a literal `.*` prefix), while a genuine prefix search still returns its user.

VULN-05 — JWT: no expiry, no algorithm pinning, weak key

OWASP	A07:2021 / A02:2021
CWE	CWE-613, CWE-757, CWE-521
Severity	High
Found by	Custom Semgrep rules <code>jwt-signed-without-expiry</code> , <code>jwt-verify-without-algorithm-pinning</code>

Four weaknesses on one trust boundary:

1. **Tokens never expired.** `jwt.sign(payload, secret)` with no options emits no `exp` claim. Combined with `localStorage` storage, a token captured once grants **permanent** access — no logout, password change or revocation can invalidate it.
2. **No algorithm pinning.** `jwt.verify` without `algorithms` honours the `alg` in the attacker-supplied token header — the basis of algorithm confusion. Especially relevant since the pinned `jsonwebtoken@8.5.1` carries advisories for exactly this.
3. **No issuer/audience binding.** Any token signed with the same secret by any other service was accepted.
4. **The signing key was never validated.** `.env.sample` shipped `JWT_SECRET=" Put ur JWT secret Here"` — 23 guessable characters. A short secret is offline-brute-forceable from a single captured token, after which the attacker mints valid tokens for **any** user ID.

A fifth, non-cryptographic defect sat in the same middleware: the user lookup called `next()` with no null check and no `.catch()`. A token for a deleted account set `req.user = null` and continued, so controllers dereferenced `null._id` — an unauthenticated 500 on every protected route.

The fix. `server/config/jwt.config.js` centralises the token contract so signing and verification cannot drift: HS256 pinned both sides, `expiresIn: "1h"`, issuer/audience enforced, and `assertSecretIsStrong()` called at boot, which **refuses to start** if `JWT_SECRET` is missing, under 32 characters, or still the sample placeholder. The middleware now requires the `Bearer` scheme, wraps verification in `try/catch`, and rejects tokens whose subject no longer exists.

Verified. Decoded live tokens carry `exp`, `alg: HS256`, the expected `iss` and `aud`; forged tokens and missing headers return `401`; the boot check rejects unset, 6-character and placeholder secrets.

VULN-06 — Password reset: broken, and insecure

OWASP	A07:2021
CWE	CWE-640, CWE-522, CWE-204, CWE-521
Severity	High
Found by	Manual code review

Four defects, one of which meant **the feature never worked**:

1. **The new password was never saved.**

```
user.password = HashPwd; // lower-case "p" - schema field is "Password"
```

Mongoose silently ignores assignment to an unknown path. The hash was discarded while the code cleared the reset token and replied *"Password Updated successfully"*.

The security consequence exceeds the functional one: a user resetting **because they believe they are compromised** is told the rotation succeeded, keeps their old credentials, and burns their only token. The application actively misled the user about their own account's state.

2. **Reset tokens stored in plaintext** — any read of the collection (backup, replica, injection, insider) yielded working reset credentials.
3. **Account-existence oracle** — unknown addresses returned `"No User exists with that email"` while known ones returned `"check your Email Inbox"`.
4. **No password policy** — `signup` and `newPwd` accepted `"a"`. A bcrypt cost of 12 is irrelevant when the plaintext is one character.

The fix. `user.Password = HashPwd`; tokens hashed with SHA-256 before storage and looked up by digest; tokens cleared on use (single-use); both branches return one identical message; and a shared policy in `server/utils/validators.js` (10–128 chars, mixed case + digit) applied to **both** `signup` and `newPwd`.

Why SHA-256 is correct here but wrong for passwords. Reset tokens are 256 bits of `crypto.randomBytes` — not dictionary-attackable, so the digest need only be preimage-resistant. Passwords are low-entropy and human-chosen, which is why they need a *slow, salted* KDF like bcrypt. Same primitive choice, opposite answers, because the threat differs.

Verified. Policy rejects `"a"`, `"password"`, `"Password1"`, `"Sh0rt"`; known and unknown addresses produce byte-identical reset responses.

VULN-07 — No rate limiting anywhere

OWASP	A07:2021
CWE	CWE-307, CWE-799
Severity	High
Found by	Manual review

Impact. Unlimited online password guessing against `/signin` (no lockout, no delay, no CAPTCHA, no alerting) — compounded by the absent password policy of VULN-06. `/reset-pwd` could be called without bound, making it both an enumeration tool and a **mail bomb** against a third party using this application's sender reputation. `/new-pwd` could be used to grind tokens.

The fix. Three tiers in `server/middleware/rateLimit.middleware.js`:

Limiter	Budget	Applied to
<code>authLimiter</code>	10 / 15 min, <code>skipSuccessfulRequests</code>	<code>/signin</code> , <code>/signup</code> , <code>/new-pwd</code>
<code>passwordResetLimiter</code>	5 / hour	<code>/reset-pwd</code> (each call sends mail)
<code>globalLimiter</code>	500 / 15 min	whole API

The proxy subtlety. A rate limiter is only as trustworthy as the client IP it keys on. Setting `trust proxy` to a blanket `true` makes Express believe the left-most `X-Forwarded-For` value — which the **client supplies** and can rotate at will, converting the limiter into decoration. So `trust proxy` is now an explicit, validated deployment decision (`TRUST_PROXY=<hop count>`), defaulting to `false`.

Verified. Repeated failed logins return `429` — *after the correction in VULN-11, without which this control did not work at all.*

VULN-08 — Unrestricted upload and absent input validation

OWASP	A04:2021 / A03:2021
CWE	CWE-434, CWE-20, CWE-400
Severity	High
Found by	Manual review

The flaws.

1. **Images accepted on trust.** A base64 blob plus a *client-declared* `photoType`, stored verbatim and rendered back to every viewer as `src={`data:${item.PhotoType};base64,${item.Photo}`}` . No check of content, size or type.
2. **`createPost` crashed on every call.** The body destructured `photoEncode` but the code read an undeclared `photoEncoded` — a `ReferenceError`, so post creation returned 500 every time.
3. **No length limits** on Title, Body or comments.
4. **No ObjectId validation** anywhere.
5. **`$push` for Followers/Following/Bookmarks.** Replaying a follow appended a duplicate each time, so one caller could grow *another user's* array until it hit MongoDB's 16 MB document ceiling and permanently broke that profile.
6. `express.json({ limit: "50mb" })` — 50 MB buffered per request.

The fix. `server/utils/imageUpload.js` validates by **content, not claim**: magic-byte sniffing for JPEG/PNG/GIF/WebP, the *sniffed* type is what gets stored, a declared type is only a cross-check, 5 MB decoded ceiling checked before allocation, and strict base64 validation. Applied to both `createPost` and `updatePicture`. Plus: the `ReferenceError` fixed, text bounds (150/2200/1000), `isValidObjectId` guards on eight endpoints, `$push` → `$addToSet`, self-follow rejected, body limit 50 MB→8 MB.

SVG is deliberately excluded from the allow-list: it is an XML document that can carry `<script>`, so serving user-supplied SVG from the app's origin would be a genuine stored-XSS primitive.

Honest assessment of exploitability. A spoofed `PhotoType` such as `text/html` does **not** yield script execution *in this client*, because the value only ever reaches an `<img src>` or a CSS `background-image`, and neither executes HTML. This is therefore reported as content-type spoofing and unrestricted upload (CWE-434), **not** as stored XSS. It still warrants fixing — the bytes are served to every viewer, the API is not bound to this one React client, and a future consumer that renders the field differently turns it into an execution primitive. Overstating it as "stored XSS" would have been easy and would have been wrong.

Verified. 15 validator cases plus live probes: HTML-declared-as-PNG rejected, SVG rejected, 6 MB payload rejected, `{ $ne: null }` rejected, mismatched declared type rejected, genuine JPEG/PNG/GIF/WebP accepted.

VULN-09 — Information disclosure via errors; missing headers

OWASP	A05:2021
CWE	CWE-209, CWE-248, CWE-1021
Severity	Medium
Found by	Custom Semgrep rule <code>raw-error-object-in-response</code> (6 hits) + manual review

The flaws. Eleven handlers did `res.status(422).json({ error: err })`, serialising raw Mongoose errors — collection names, field names, driver version, schema shape — to anyone who sent a malformed ID. Express's default handler was in play (`NODE_ENV=development` in the shipped start script), so any unhandled throw returned an HTML **stack trace with absolute filesystem paths**. `deletePost` **never responded** to a non-owner — the ownership check had no `else`, so the request fell off the end of the function and the client hung until timeout. CORS was hard-coded to `localhost:3000`. `helmet()` ran with bare defaults. Feeds returned every post with images inlined, unbounded.

The fix. Generic client messages with server-side logging and meaningful status codes; explicit JSON 404; a centralised error handler replacing Express's stack-trace default; Helmet configured explicitly (deny-all CSP — appropriate since this origin serves only JSON — `no-referrer`, HSTS under production, CORP `same-site`); CORS from an allow-list (`CORS_ORIGINS`); `x-powered-by` disabled *before* all middleware; feeds capped at 50.

A latent crash was also fixed: `Photo` defaults to the **string** `"no photo"`, so the unguarded `item.Photo.toString("base64")` corrupted any feed containing an image-less post. A single guarded `serializePost` helper now handles it.

Incidentally but necessarily, all remaining Mongoose **callbacks** were converted to `async/await` — callback support was *removed* in Mongoose 7, so this was a prerequisite for VULN-10.

Verified. No `X-Powered-By`; CSP, `X-Content-Type-Options: nosniff`, frame-ancestors and `Referrer-Policy` all present; unknown routes return JSON 404; malformed delete IDs return 400 instead of hanging.

VULN-10 — 32 vulnerable server dependencies

OWASP	A06:2021
CWE	CWE-1035
Severity	High
Found by	<code>npm audit</code>

32 advisories (2 critical, 17 high). Several sat **directly on the authentication path**:

- `jsonwebtoken@8.5.1` — "signature validation bypass due to insecure default algorithm", and via `jws`, "improperly verifies HMAC signature". This is the library guarding every protected route, and the missing algorithm pinning in VULN-05 is exactly the configuration these advisories punish.

- `mongoose@6.0.5` — five prototype-pollution advisories plus "Mongoose search injection" and "improper sanitization of `$nor`" — the same class fixed at application level in VULN-03.
- `express` (XSS via `response.redirect()`, open redirect), `qs`, `minimist`, `path-to-regexp` (ReDoS), `body-parser`.

The tree was also literally unrunnable on a supported Node release. `jsonwebtoken@8.5.1` depends on `buffer-equal-constant-time`, which reads `SlowBuffer.prototype`. `SlowBuffer` was removed in Node 24, so on Node 25 merely *requiring* the auth controller threw:

```
TypeError: Cannot read properties of undefined (reading 'prototype')
```

The application could therefore only run on a Node version that is itself end-of-life and no longer receiving security patches — a supply-chain problem that **forced an insecure runtime**.

The fix. `express` 4.17→4.22, `mongoose` 6.0→8.24, `jsonwebtoken` 8.5→9.0, `helmet` 4.6→8.3, `dotenv` 10→16, `@sendgrid/mail` 7→8, `nodemon` 2→3, then `npm audit fix`.

Result: 32 → 0. All modules load on Node 25.

VULN-11 — HTTP 200 on auth failure silently disabled the rate limiter

OWASP	A07:2021
CWE	CWE-307
Severity	High
Found by	The runtime verification harness

This defect was in our own fix for VULN-07, not in the upstream code.

The upstream controllers signalled failure with HTTP **200** plus an error field:

```
return res.json({ error: "Invalid Email or Password" });
```

The limiter was configured with `skipSuccessfulRequests: true`, which decides what counts as "successful" **from the HTTP status code**. Because failed logins returned 200, the limiter classified brute-force attempts as *successes* and skipped them. The harness proved it: **14 consecutive wrong passwords, no 429**.

So the brute-force protection reported under VULN-07 **did not exist**. The configuration looked correct and was inert.

The fix. Accurate status codes — 400 (malformed/policy), 401 (bad credentials), 409 (duplicate), 429 (limited) — which is both correct HTTP semantics and the precondition for `skipSuccessfulRequests` to mean what it says. The four client auth screens were updated to read `err.response.data.error`, since axios rejects on non-2xx and they previously routed these into a `.catch` that only did `console.log`.

The lesson. A security control that is present, plausible-looking and **inert** is more dangerous than an absent one, because it stops anyone looking further. This was caught only by testing against a running server. *Reading the diff would never have found it* — the configuration was correct in isolation and wrong in context.

Verified. Repeated failed logins now return `429` after 8 attempts.

VULN-12 — 200 client advisories; unpinned GitHub dependency; no client CSP

OWASP	A06:2021 / A03:2021
CWE	CWE-1035, CWE-1104
Severity	High
Found by	<code>npm audit</code> + dependency review

200 advisories (18 critical), including arbitrary code execution in `@babel/traverse`, private-key extraction in `elliptic`, template injection in `ejs`.

Two problems were **not advisories at all**:

- `materialize-css` — a dependency **not used anywhere** in the source (its only trace is a commented-out CDN `<link>`), yet carrying a live XSS advisory. Unused dependencies are pure liability.
- `react-reactions` — declared as `"github:randomdipesh/react-reactions"`: an **unpinned dependency on an arbitrary individual's GitHub fork**, with no version, no integrity hash and no registry review. Whoever controls that repository controls what this build installs. It was also unused. `npm audit` would never flag this, since it only reports advisories against published packages.

The fix. `react-scripts` 4.0.3→5.0.1, `axios` 0.21.4→1.13.2 (the only runtime dependency with advisories), both unused packages removed, then `npm audit fix`. **200 → 26, all criticals gone.**

A client-side CSP was added to `client/public/index.html` (`script-src 'self'`, `object-src 'none'`, `base-uri 'self'`, `form-action 'self'`, pinned `connect-src`) — the API's CSP only governs the API origin, and the SPA had no policy at all. This required `INLINE_RUNTIME_CHUNK=false`, since CRA otherwise inlines its webpack runtime and the strict policy would block it, rendering a blank page. **Verified: the production build contains zero inline scripts, and compiles successfully.**

4. Vulnerabilities NOT fixed, and why

Reporting these honestly matters more than an artificially clean sheet.

4.1 Twenty-six client build-toolchain advisories

Not fixed. All 26 are in `react-scripts`, `webpack`, `webpack-dev-server`, `jest`, `jsdom`, `postcss`, `svgo`, `workbox` and `rollup`. **None appear in the production bundle served to users.** They are reachable only by someone who can already run the build — a different and much narrower threat model.

Clearing them means abandoning Create React App for Vite: a migration whose blast radius vastly exceeds this assignment, rewriting files unrelated to any vulnerability. **Accepted residual risk**, with the mitigation that the build runs only on developer machines and CI.

4.2 React held at version 17

Not fixed. Material-UI v4, which every screen is built on, is incompatible with React 18's rendering model. Upgrading React means migrating the entire UI to MUI v5 — a change unrelated to security that would swamp the actual vulnerability fixes in the diff.

4.3 JWT still stored in `localStorage`

Not fixed. `localStorage` is readable by any JavaScript on the origin, so an XSS gives up the session token. An `httpOnly` cookie would be immune.

Mitigations in place: tokens now expire in 1 hour (VULN-05, previously never), and a strict CSP (VULN-12) substantially reduces XSS reachability. The change touches every authenticated request in the client and needs CSRF protection added alongside — cookies are sent automatically, so moving to cookies *introduces* a CSRF exposure that Bearer headers do not have. Rushing it would trade one vulnerability class for another. **Accepted residual risk**; recommended as the next piece of work.

4.4 Rate-limit counters are in-process

Partially fixed. `express-rate-limit`'s default store is per-process memory, so counters reset on restart and are not shared between instances. A multi-instance deployment needs `rate-limit-redis`. Correct for a single-instance deployment; documented rather than silently assumed.

4.5 OAuth state store is in-process

Partially fixed. Same reasoning (`utils/oauthStore.js`). Critically, it keeps the PKCE `code_verifier` **server-side**, which is the security-relevant property; persistence is an availability concern, not a confidentiality one.

4.6 `style-src 'unsafe-inline'`

Not fixed. Material-UI v4 injects component styles at runtime via JSS, so the CSP cannot forbid inline styles without breaking every screen. Inline *styles* are a substantially lower risk than inline *scripts*, which **are** blocked. Resolved by the same MUI v5 migration as §4.2.

4.7 `frame-ancestors` in a `<meta>` CSP

Not fixable there. `frame-ancestors` is ignored in a meta policy and must be sent as a real header by whatever serves the static build. The API already sets it for its own responses. Documented as a deployment requirement.

4.8 Email delivery is disabled

Pre-existing. The SendGrid calls are commented out upstream, so reset emails are not actually sent. The reset flow is otherwise fully fixed and hardened; enabling delivery is a matter of supplying an API key.

5. OAuth 2.0 / OpenID Connect implementation

Full detail in `OAUTH.md`; summarised here.

5.1 What was added

The application had exactly one sign-in method: email and password. Added: **Sign in with Google**, using the **Authorization Code grant with PKCE** and OpenID Connect for identity. The password flow is **not removed** — existing accounts keep working, and an account can link both.

5.2 Why this grant type

Grant	Verdict
Implicit	Rejected — deprecated by RFC 9700. Returns tokens in the URL fragment, reaching history, <code>Referer</code> headers and logs.
Resource Owner Password Credentials	Rejected — deprecated, and would require this app to handle <i>Google</i> passwords directly, the exact thing OAuth prevents.
Client Credentials	Rejected — authenticates a service, not a person. No user identity.
Authorization Code + PKCE	Chosen. Code crosses the front channel; tokens are exchanged server-to-server. PKCE binds the code to the client that began the flow.

PKCE matters even for a confidential client with a secret: it defends against authorization-code interception. An attacker who obtains the `code` still cannot redeem it without the `code_verifier`, which never leaves the server. Only `S256` is used — `plain` gives no protection against an observer.

5.3 Design decision: the one-time exchange code

The obvious shortcut is `redirect(client + "?token=" + jwt)`. That writes a **live session token into a URL** — and URLs reach browser history, proxy and server access logs, and the `Referer` header of every later request.

Instead the API mints an **opaque, single-use, 60-second code**. The SPA redeems it over `POST` and it is deleted on first use, so the JWT never appears in a URL. The callback screen also strips the code from the address bar via `history.replaceState` on load.

5.4 Security controls

Control	Purpose
<code>state</code> parameter	CSRF — an attacker cannot forge an accepted callback
<code>state</code> mirrored in an <code>httpOnly</code> , <code>SameSite=Lax</code> cookie	Binds the callback to the browser that started the flow
<code>crypto.timingSafeEqual</code> comparison	The state check cannot be probed by timing
Single-use authorization request	A replayed callback fails
PKCE <code>S256</code> , verifier never leaves the server	An intercepted code cannot be redeemed
ID token signature verified against Google's JWKS	Decode-and-trust — common in tutorial code — would let anyone mint an identity
<code>iss</code> / <code>aud</code> / <code>exp</code> validated	Rejects tokens minted for another application
<code>email_verified</code> required	Prevents takeover by registering a Google account on someone else's address
Linked on <code>sub</code> , not email	<code>sub</code> is immutable; email addresses can be reassigned between people
Google tokens not reused as session tokens	The app mints its own JWT with its own expiry and claims (VULN-05)
Rate limited	Same <code>authLimiter</code> as other credential endpoints
No password generated for Google users	A generated-then-discarded credential exists, can be guessed, and protects nothing

5.5 Verification

`security/verify-oauth.js` — **23 checks, all passing**: grant type, PKCE method and challenge format, requested scopes, state cookie flags, constant-time CSRF rejection (missing / mismatched / unknown state), consent-denial handling, exchange-code validation, and per-request freshness.

6. Practices that would have prevented these defects

Each recommendation is tied to the specific finding it would have stopped.

Practice	Would have prevented
Secure defaults in the data layer — mark secrets <code>select: false</code> when the field is <i>created</i>	VULN-02. The leak existed because secrets were public-by-default and the projection was a deny-list.
Allow-lists, not deny-lists, at every boundary	VULN-02, VULN-08. A deny-list's failure mode is <i>exposure</i> ; an allow-list's is <i>omission</i> . Choose the direction whose failure is harmless.
Validate types at the trust boundary (schema validation: Joi/Zod/ <code>express-validator</code>)	VULN-03, VULN-08. One declarative schema per endpoint makes "is this a string?" structural rather than a thing to remember.
Centralise cross-cutting concerns — one <code>router</code> with auth applied by default, opting <i>out</i> for public routes	VULN-01. The bug was a <i>missing argument</i> on one line out of seventeen — invisible in review, impossible if auth were the default.
Automated dependency scanning in CI (<code>npm audit</code> / Dependabot as a build gate)	VULN-10, VULN-12. 232 advisories accumulated because nothing ever failed a build.
Ban unpinned VCS dependencies ; audit for unused ones	VULN-12. <code>github:randomdipesh/react-reactions</code> is unreviewable and mutable; <code>materialize-css</code> was pure unused liability.
Integration tests for security controls, not just unit tests	VULN-11. The decisive one. The limiter was correctly configured and totally inert. Only a test that <i>actually sent 14 bad logins</i> could find that.
Correct HTTP semantics	VULN-11. Returning 200 for failure broke a downstream control that reasonably trusted status codes.
Fail fast on unsafe configuration at boot	VULN-05. A placeholder <code>JWT_SECRET</code> should stop the process, not silently sign tokens.
Treat linter and type warnings as errors	VULN-08's <code>ReferenceError</code> (<code>photoEncoded</code> vs <code>photoEncode</code>) and VULN-06's <code>user.password</code> vs <code>user.Password</code> are both <i>typos</i> that TypeScript or <code>no-undef</code> would have caught at author time. Two of the most serious findings were spelling mistakes.
Threat-model the recovery flow, not just the login flow	VULN-02, VULN-06. Password reset is an <i>alternative authentication path</i> and deserves the same scrutiny as the primary one. It got far less.
Security-focused code review with a checklist	Most of the above. Generic review asks "does it work?"; security review asks "what happens if this input is an object?"

6.1 The overarching lesson

Tooling is necessary but nowhere near sufficient. The community Semgrep rulesets — four of them, 292 rules — found **zero** issues in an application containing two critical vulnerabilities. `npm audit` found 232 real problems but could not see a single one of the logic flaws. Manual review found the broken reset flow that no tool flagged. Runtime testing found VULN-11, which was invisible in the source.

Each technique has a characteristic blind spot, and those blind spots are not random — they are *structural*. Defence in depth applies to your **assessment methodology** just as much as to your controls.

7. Conclusion

Twelve distinct vulnerabilities were identified and fixed, exceeding the required seven. Two were critical and chained into pre-authentication account takeover. An OAuth 2.0 / OpenID Connect sign-in flow was added using the Authorization Code grant with PKCE, with ID-token signature verification and CSRF protection.

Every fix is a single commit documenting the OWASP category, CWE, severity, discovery method, exploit path, remediation and verification. All fixes are verified by automated checks against a running server: **30/30** security checks and **23/23** OAuth checks.

Seven items remain unfixed or partially fixed. Each is documented in §4 with its reasoning and residual risk, because a report that claims everything was solved is less useful — and less credible — than one that says exactly where the boundaries are.

Appendix A — Evidence

File	Contents
<code>security/semgrep-rules.yml</code>	Custom Semgrep ruleset (7 rules)
<code>security/reports/semgrep-community-before.txt</code>	Community rulesets: 0 findings
<code>security/reports/semgrep-custom-before.json</code>	Custom rules: 20 findings (pre-tuning)
<code>security/reports/semgrep-custom-after.txt</code>	Custom rules after fixes + tuning: 0 findings
<code>security/reports/npm-audit-server-before.*</code>	32 advisories
<code>security/reports/npm-audit-server-after.txt</code>	0 advisories
<code>security/reports/npm-audit-client-before.*</code>	200 advisories
<code>security/reports/npm-audit-client-after.txt</code>	26 advisories (build-only)
<code>security/verify-fixes.js</code> + <code>reports/verify-fixes-after.txt</code>	30/30 passing
<code>security/verify-oauth.js</code> + <code>reports/verify-oauth-after.txt</code>	23/23 passing

Appendix B — Commit map

Commit	Content
<code>Baseline: unmodified upstream</code>	Pristine original — every later change is attributable
<code>chore(security): scanning toolchain</code>	Custom rules + pre-fix evidence
<code>fix(VULN-01 ... VULN-12)</code>	One commit per vulnerability
<code>feat(oauth): Sign in with Google</code>	OAuth 2.0 + PKCE implementation

```
git log --oneline          # overview
git log --grep="VULN-02" -p # one vulnerability end to end
```